Datapath

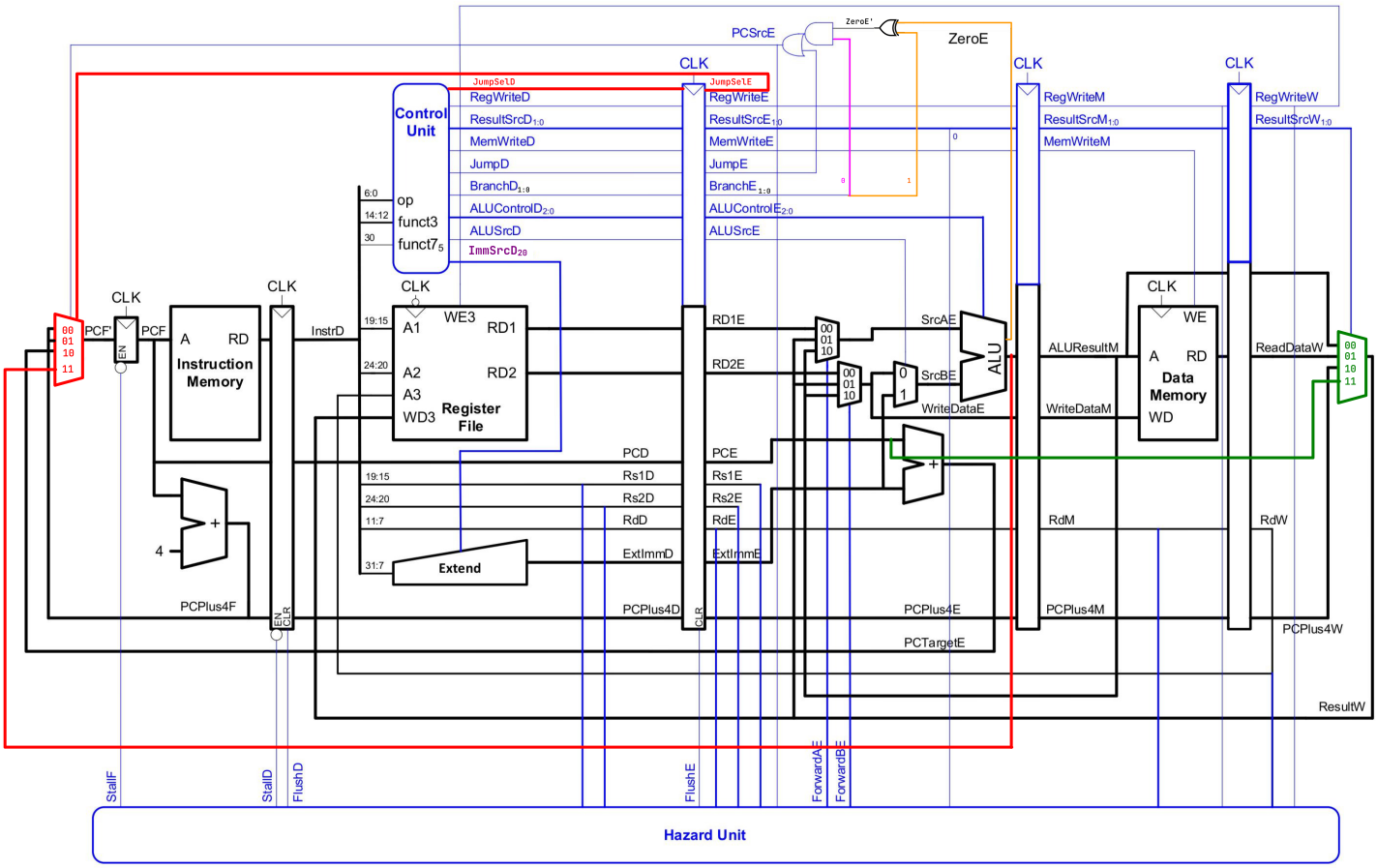


Figure 1: Datapath Diagram

Controller

Instr	Type	opcode	funct3	funct7	ImmSrc	ALUSrc	ALUControl	Branch	Jump	MemWrite	ResultSrc	RegWrite	JumpSel
add	R	0110011	000	0000000	—	0	ADD	00	0	0	00	1	X
sub	R	0110011	000	0100000	—	0	SUB	00	0	0	00	1	X
and	R	0110011	111	0000000	—	0	AND	00	0	0	00	1	X
or	R	0110011	110	0000000	—	0	OR	00	0	0	00	1	X
slt	R	0110011	010	0000000	—	0	SLT	00	0	0	00	1	X
addi	I	0010011	000	—	I(000)	1	ADD	00	0	0	00	1	X
xori	I	0010011	100	—	I(000)	1	XOR	00	0	0	00	1	X
ori	I	0010011	110	—	I(000)	1	OR	00	0	0	00	1	X
slti	I	0010011	010	—	I(000)	1	SLT	00	0	0	00	1	X
lw	I	0000011	010	—	I(000)	1	ADD	00	0	0	01	1	X
sw	S	0100011	010	—	S(001)	1	ADD	00	0	1	00	0	X
beq	B	1100011	000	—	B(010)	0	SUB	01	0	0	00	0	X
bne	B	1100011	001	—	B(010)	0	SUB	11	0	0	00	0	X
jal	J	1101111	—	—	J(011)	X	ADD	00	1	0	10	1	0
jalr	I	1100111	000	—	I(000)	1	ADD	00	1	0	10	1	1
lui	U	0110111	—	—	U(100)	1	PASS_B	00	0	0	00	1	X

Table 1: Control Signals

Code Implementation

```c
#include <stdio.h>

int main() {
    int array[10] = {5, 12, -1, 16, -2, 3,
     8, 0, 9, 20};

    int min_val = array[0];

    for (int i = 1; i < 10; i++) {
        int current_val = array[i];

        if (current_val < min_val) {
            min_val = current_val;
        }
    }

    printf("The minimum value is: %d\n",
     min_val);

    return 0;
}
```

C Source

→

```
start:
    addi x10, x0, 0
    addi x11, x0, 9
    lw   x12, 0(x10)
    addi x10, x10, 4

loop:
    beq  x11, x0, finish
    lw   x13, 0(x10)
    slt  x14, x13, x12
    beq  x14, x0, skip_update
    add  x12, x13, x0

skip_update:
    addi x10, x10, 4
    addi x11, x11, -1
    jal  x0, loop

finish:
    addi x15, x0, 400
    sw   x12, 0(x15)
```

RISC-V

→

```
00000513
00900593
00052603
00450513
02058063
00052683
00c6a733
00070463
00068633
00450513
fff58593
fe5ff06f
19000793
00c7a023
```

Machine Code

Testcase

```
5
12
-1
16
-2
3
8
0
9
20
```

Input Data

⇓

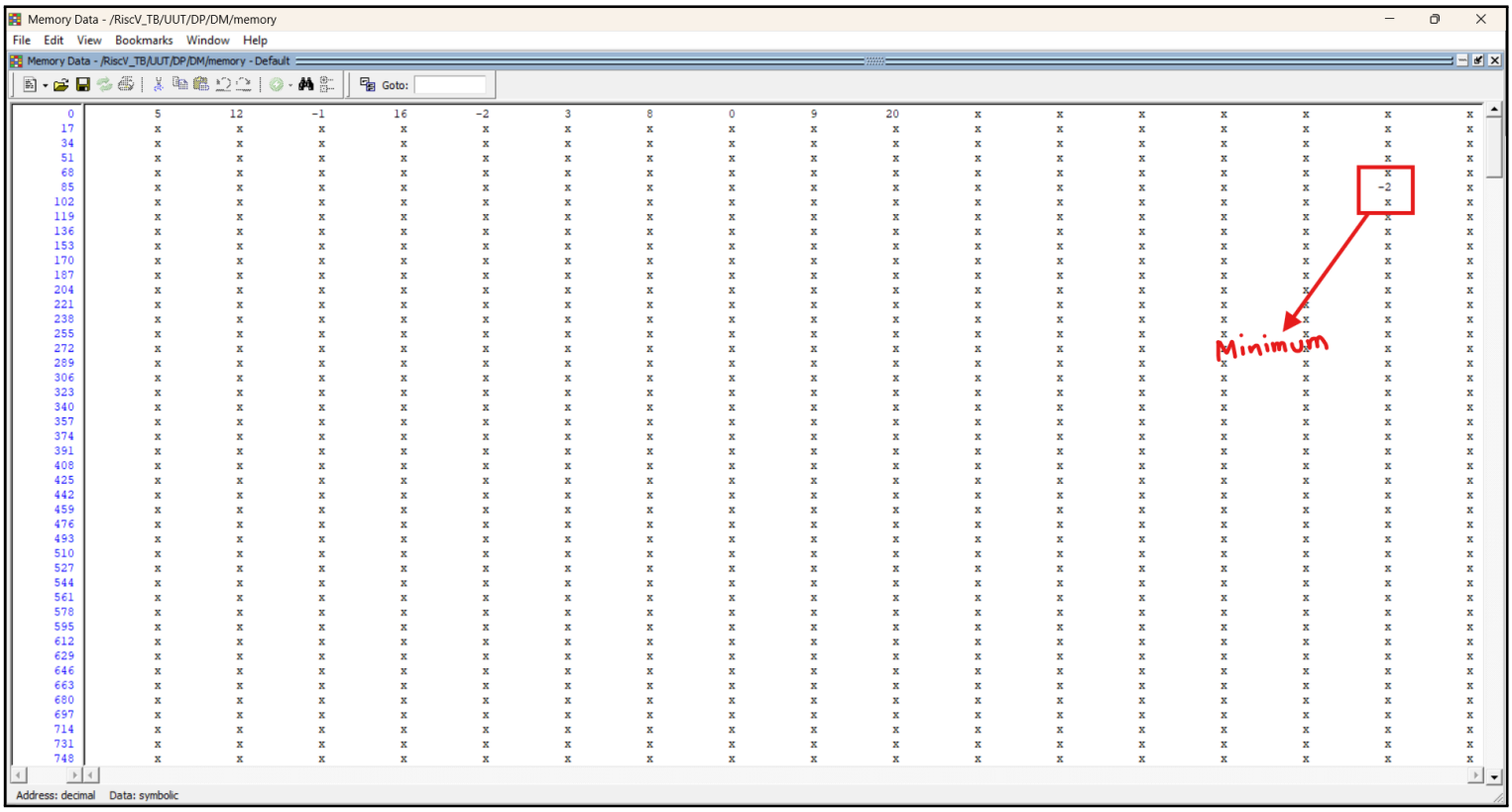
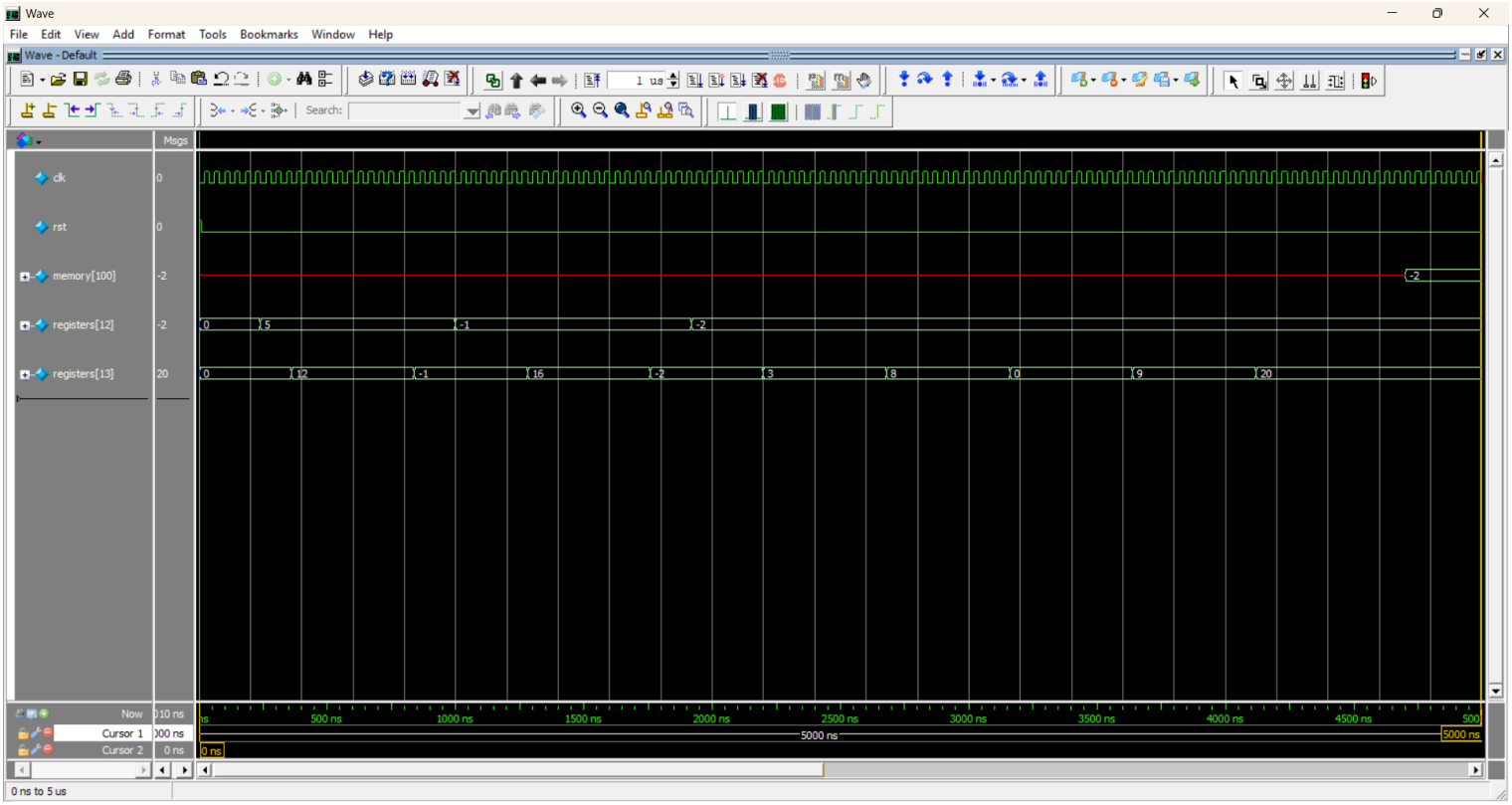


Figure 2: Output Data

Waveform



- **registers[12]** (x_{12}) is for minimum variable.
- **registers[13]** (x_{13}) is for arr[i]
- **memory[200]** is where the final answer (The minimum of the array) saves.

RISC-V Reference

James Zhu <jameszhu@berkeley.edu>

RISC-V Instruction Set

Core Instruction Formats

31 27	26 25	24 20	19 15	14 12	11 7	6 0	
funct7		rs2	rs1	funct3	rd	opcode	R-type
imm[11:0]			rs1	funct3	rd	opcode	I-type
imm[11:5]		rs2	rs1	funct3	imm[4:0]	opcode	S-type
imm[12|10:5]		rs2	rs1	funct3	imm[4:1|11]	opcode	B-type
imm[31:12]					rd	opcode	U-type
imm[20|10:1|11|19:12]					rd	opcode	J-type

RV32I Base Integer Instructions

Inst	Name	FMT	Opcode	funct3	funct7	Description (C)	Note
add	ADD	R	0110011	0x0	0x00	rd = rs1 + rs2	
sub	SUB	R	0110011	0x0	0x20	rd = rs1 - rs2	
xor	XOR	R	0110011	0x4	0x00	rd = rs1 ^ rs2	
or	OR	R	0110011	0x6	0x00	rd = rs1 | rs2	
and	AND	R	0110011	0x7	0x00	rd = rs1 & rs2	
sll	Shift Left Logical	R	0110011	0x1	0x00	rd = rs1 << rs2	
srl	Shift Right Logical	R	0110011	0x5	0x00	rd = rs1 >> rs2	
sra	Shift Right Arith*	R	0110011	0x5	0x20	rd = rs1 >> rs2	msb-extends
slt	Set Less Than	R	0110011	0x2	0x00	rd = (rs1 < rs2)?1:0	
sltu	Set Less Than (U)	R	0110011	0x3	0x00	rd = (rs1 < rs2)?1:0	zero-extends
addi	ADD Immediate	I	0010011	0x0		rd = rs1 + imm	
xori	XOR Immediate	I	0010011	0x4		rd = rs1 ^ imm	
ori	OR Immediate	I	0010011	0x6		rd = rs1 | imm	
andi	AND Immediate	I	0010011	0x7		rd = rs1 & imm	
slli	Shift Left Logical Imm	I	0010011	0x1	imm[5:11]=0x00	rd = rs1 << imm[0:4]	
srli	Shift Right Logical Imm	I	0010011	0x5	imm[5:11]=0x00	rd = rs1 >> imm[0:4]	
srai	Shift Right Arith Imm	I	0010011	0x5	imm[5:11]=0x20	rd = rs1 >> imm[0:4]	msb-extends
slti	Set Less Than Imm	I	0010011	0x2		rd = (rs1 < imm)?1:0	
sltiu	Set Less Than Imm (U)	I	0010011	0x3		rd = (rs1 < imm)?1:0	zero-extends
lb	Load Byte	I	0000011	0x0		rd = M[rs1+imm][0:7]	
lh	Load Half	I	0000011	0x1		rd = M[rs1+imm][0:15]	
lw	Load Word	I	0000011	0x2		rd = M[rs1+imm][0:31]	
lbu	Load Byte (U)	I	0000011	0x4		rd = M[rs1+imm][0:7]	zero-extends
lhu	Load Half (U)	I	0000011	0x5		rd = M[rs1+imm][0:15]	zero-extends
sb	Store Byte	S	0100011	0x0		M[rs1+imm][0:7] = rs2[0:7]	
sh	Store Half	S	0100011	0x1		M[rs1+imm][0:15] = rs2[0:15]	
sw	Store Word	S	0100011	0x2		M[rs1+imm][0:31] = rs2[0:31]	
beq	Branch ==	B	1100011	0x0		if(rs1 == rs2) PC += imm	
bne	Branch !=	B	1100011	0x1		if(rs1 != rs2) PC += imm	
blt	Branch <	B	1100011	0x4		if(rs1 < rs2) PC += imm	
bge	Branch ≥	B	1100011	0x5		if(rs1 >= rs2) PC += imm	
bltu	Branch < (U)	B	1100011	0x6		if(rs1 < rs2) PC += imm	zero-extends
bgeu	Branch ≥ (U)	B	1100011	0x7		if(rs1 >= rs2) PC += imm	zero-extends
jal	Jump And Link	J	1101111			rd = PC+4; PC += imm	
jalr	Jump And Link Reg	I	1100111	0x0		rd = PC+4; PC = rs1 + imm	
lui	Load Upper Imm	U	0110111			rd = imm << 12	
auipc	Add Upper Imm to PC	U	0010111			rd = PC + (imm << 12)	

Figure 3: RISC-V Instruction Set